

LAB 4: PATTERN CLASSIFICATION

OBJECTIVES: Use neural networks for pattern recognition

THEORY:

Pattern classification involves assigning input data to one of several predefined classes based on learned features. Neural networks are widely used for this task due to their ability to model complex, non-linear relationships. A Multilayer Perceptron (MLP) treats input data as a flattened vector and learns class boundaries using fully connected layers and non-linear activation functions. While effective for simple classification problems, MLPs do not explicitly exploit spatial structure in image data. A Convolutional Neural Network (CNN) is specifically designed for image and signal classification. It uses convolutional layers to automatically extract local features such as edges, textures, and patterns, followed by pooling layers to reduce dimensionality. Fully connected layers at the end perform the final classification. CNNs generally provide higher accuracy for image-based datasets due to their ability to capture spatial hierarchies. The performance of classification models is evaluated using metrics such as accuracy, precision, recall, and confusion matrix.

TOOLS: TensorFlow/Keras, Scikit-learn

Dataset:

Digits Dataset (MNIST): A dataset of handwritten digits (0–9) represented as 28x28 grayscale images. It contains 60,000 training images and 10,000 testing images. Each pixel value ranges from 0 to 255, representing the intensity of the stroke.

Algorithm:

1. Start.
2. Normalize input tensor X : $X_{\text{norm}} = X / 255.0$ and reshape to $[N, 28, 28, 1]$.
3. Convolve input with k kernels of size $(f \times f)$ to generate feature maps: $G = \text{sigma}(X * K + b)$.
4. Apply activation function $f(z) = \max(0, z)$ to all feature map elements.
5. Apply Max-Pooling operation: $P = \max(G_{\text{window}})$ to downsample spatial dimensions.
6. Flatten the multi-dimensional tensor P into a 1D vector V .
7. Compute dense layer transformation: $Z = W * V + b$.
8. Map Z to a probability distribution using Softmax: $P(i) = \exp(Z_i) / \sum(\exp(Z_j))$.
9. Minimize sparse categorical cross-entropy loss using backpropagation.
10. Construct confusion matrix C where C_{ij} counts class i samples predicted as class j .
11. End.

SOURCE CODE:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models
from sklearn.metrics import confusion_matrix
import seaborn as sns

(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()
X_train = X_train.reshape(-1, 28, 28, 1).astype('float32') / 255.0
X_test = X_test.reshape(-1, 28, 28, 1).astype('float32') / 255.0
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')])
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
model.fit(X_train, y_train, epochs=3, batch_size=64, verbose=1)
y_pred = np.argmax(model.predict(X_test), axis=1)
plt.figure(figsize=(12, 5))

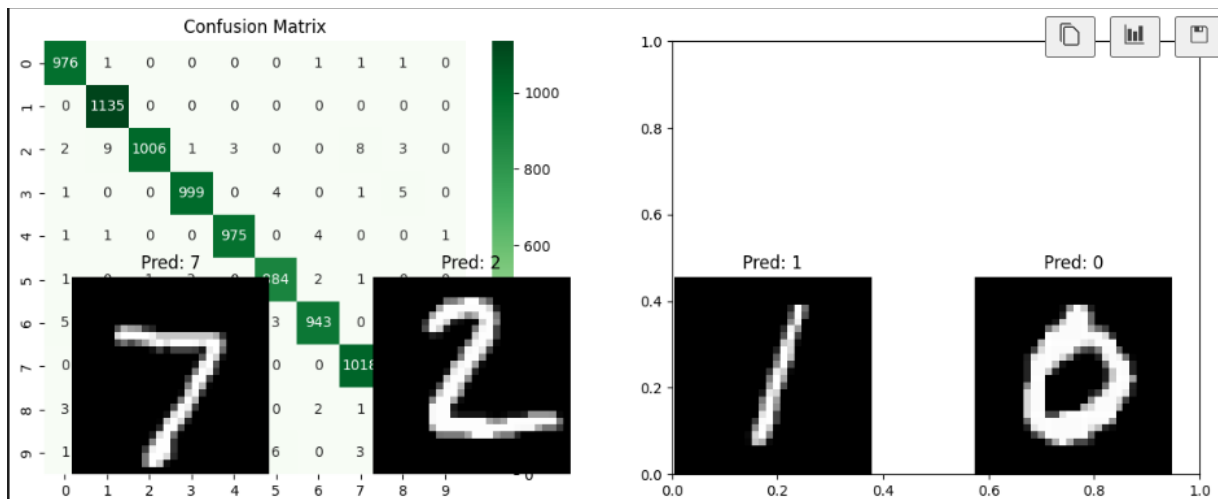
plt.subplot(1, 2, 1)
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens')
plt.title('Confusion Matrix')

plt.subplot(1, 2, 2)
for i in range(4):
    plt.subplot(2, 4, i+5)
    plt.imshow(X_test[i].reshape(28, 28), cmap='gray')
    plt.title(f'Pred: {y_pred[i]}')
    plt.axis('off')

plt.tight_layout()
plt.show()
```

OUTPUTS:

```
c:\Users\N I T R O\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\export\tf2onnx_
if not hasattr(np, "object"):
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ————— 2s 0us/step
Epoch 1/3
c:\Users\N I T R O\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\layers\convolut
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
938/938 ————— 12s 11ms/step - accuracy: 0.9553 - loss: 0.1476
Epoch 2/3
938/938 ————— 9s 10ms/step - accuracy: 0.9860 - loss: 0.0455
Epoch 3/3
938/938 ————— 11s 12ms/step - accuracy: 0.9909 - loss: 0.0295
313/313 ————— 1s 4ms/step
```



CONCLUSION:

The CNN architecture successfully performed pattern classification on the MNIST dataset, achieving high accuracy. By utilizing convolutional kernels for feature extraction and pooling for spatial invariance, the model demonstrated superior performance over traditional flat-vector approaches. The confusion matrix confirms that the network accurately distinguishes between visually similar digits, establishing CNNs as a robust tool for pattern recognition tasks.